

MediAssist

AI-Powered Healthcare Chatbot

Logic & System Design Documentation

Project	AI-Powered Healthcare Chatbot
Position	AI Engineer
Author	Hemananth U
Stack	Streamlit · Groq (llama-3.3-70b-versatile) · LangChain LCEL · ChromaDB · nomic-embed-text (Ollama)

Table of Contents

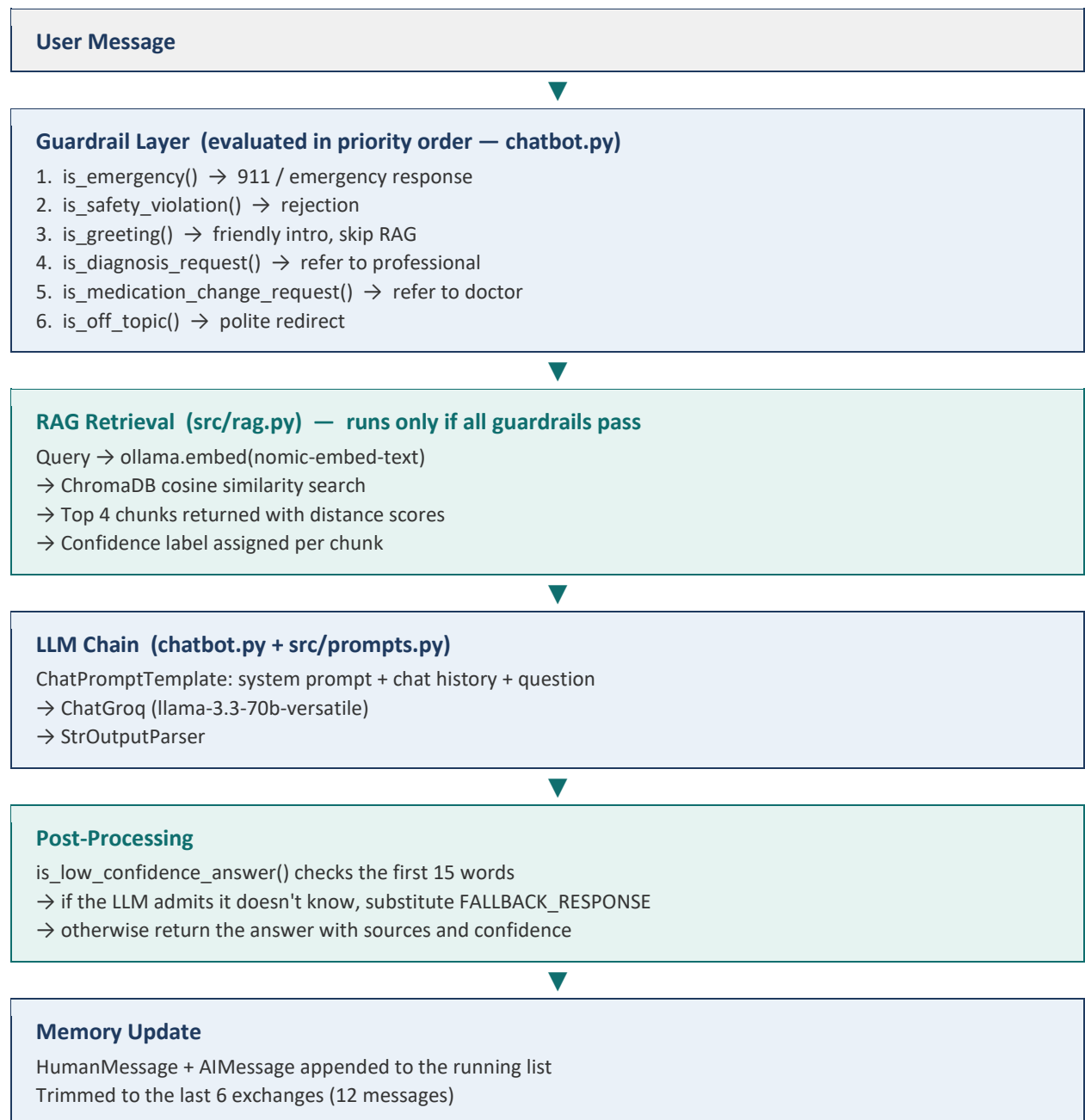
Table of Contents	2
1. How the Chatbot Processes User Queries.....	3
1.1 Full Query Processing Flow	3
1.2 Conversation Memory	4
2. Knowledge Base and Retrieval	5
2.1 Knowledge Base.....	5
2.2 Ingestion Pipeline (ingest.py)	5
2.3 Retrieval (src/rag.py)	6
3. How Prompts Are Customized.....	7
3.1 System Prompt Design.....	7
3.2 Prompt Template Structure.....	7
3.3 Temperature Control	8
4. Safety Measures and Validation	8
4.1 Guardrail System (src/utils.py)	8
4.2 Response Validation	8
4.3 Error Handling.....	9
4.4 Mandatory Disclaimer	9
5. Assumptions Made During Development.....	10
6. Tech Stack Rationale	11

1. How the Chatbot Processes User Queries

Every user message passes through a strict, sequential pipeline before a response is returned. The pipeline is organized into two major stages: Guardrail Evaluation and RAG Generation.

1.1 Full Query Processing Flow

Each incoming message moves through the following stages, in order:



**Response Returned to UI**

```
{ answer, sources, confidence, category }
```

1.2 Conversation Memory

Memory is maintained as a plain Python list of `HumanMessage` and `AIMessage` objects (LangChain Core types). The last six exchange pairs (twelve messages) are injected into the prompt through a `MessagesPlaceholder`. This gives the model enough context to handle follow-up questions without inflating token usage.

Memory is reset whenever the user clicks New Conversation in the sidebar, which calls `HealthChatbot.clear_memory()`.

2. Knowledge Base and Retrieval

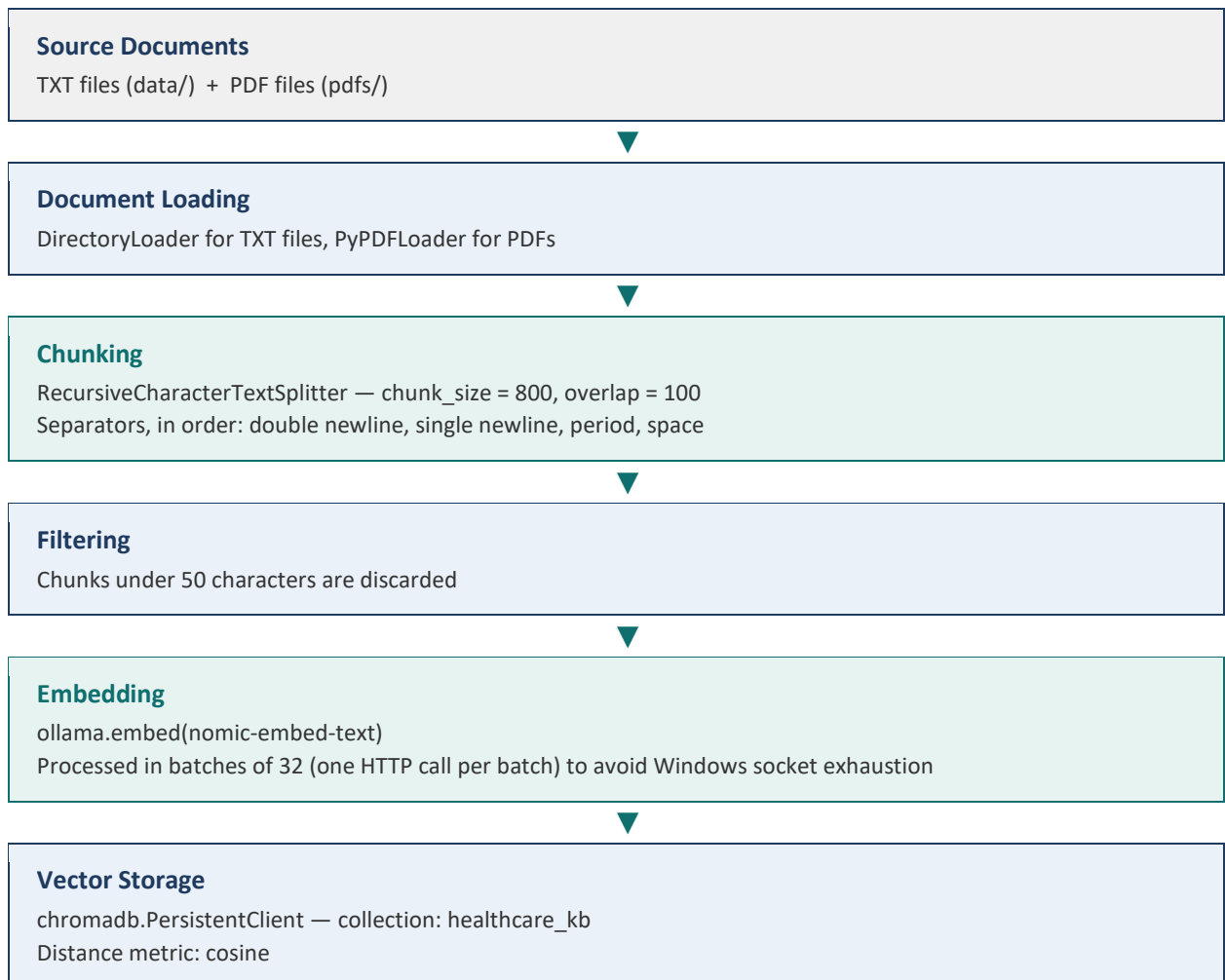
2.1 Knowledge Base

The knowledge base draws on two source types:

Source	Files	Content
TXT (curated)	symptoms_diseases.txt, nutrition_lifestyle.txt, first_aid.txt	Structured healthcare reference content written specifically for this chatbot
PDF (authoritative)	11 documents, including WHO manuals, CDC fact sheets, a hypertension guide, a type-2 diabetes guide, a mental health fact sheet, and a first aid manual	Published healthcare authority documents

In total, 615 pages and documents were processed into 2,919 chunks after splitting.

2.2 Ingestion Pipeline (ingest.py)



Ingestion is idempotent: re-running python ingest.py wipes and rebuilds the vector store cleanly using shutil.rmtree, in a cross-platform-safe way.

2.3 Retrieval (src/rag.py)

At query time, the following steps take place:

1. The user query is embedded using `ollama.embed(nomic-embed-text)` — the same model used at ingestion time, which keeps the vector space consistent.
2. ChromaDB performs a cosine similarity search and returns the top 4 chunks together with their distance scores.
3. Each chunk is assigned a confidence label based on cosine-distance thresholds.
4. The confidence label of the top-ranked chunk is used as the overall response confidence shown in the UI.

Confidence Label	Cosine Distance
High	≤ 0.25
Medium	≤ 0.45
Low	> 0.45

3. How Prompts Are Customized

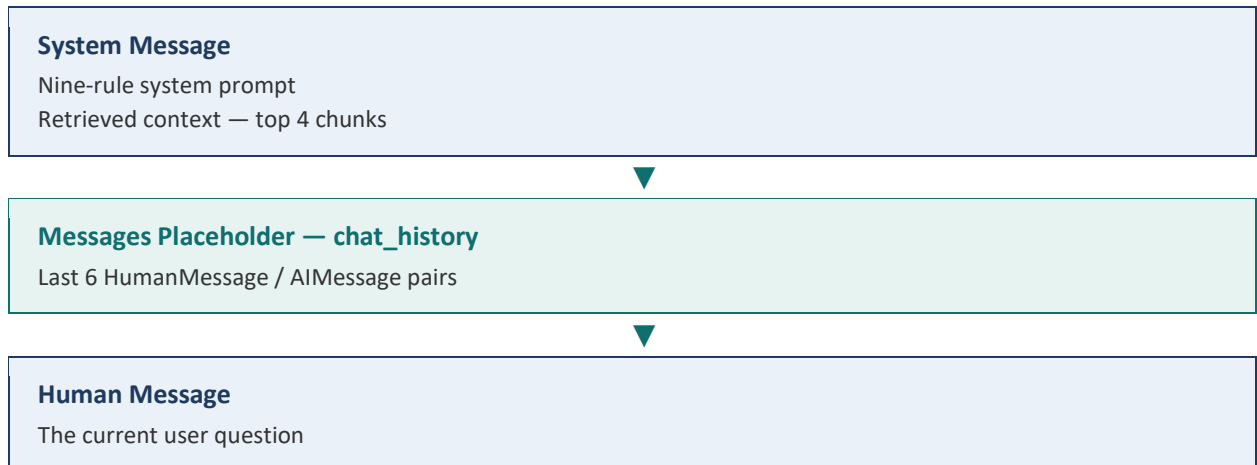
3.1 System Prompt Design

The system prompt (src/prompts.py) is injected as the first message in every LLM call. It encodes nine explicit behavioral rules:

Rule	Purpose
Answer only healthcare questions	Domain restriction
Use retrieved context as the primary source	Grounds answers in the knowledge base
Fallback phrase if context is missing	Prevents hallucination
Never diagnose diseases	Core safety constraint
Never prescribe medications	Core safety constraint
Diagnosis-request response	Explicit wording for refusal
Medication-change response	Explicit wording for refusal
Keep answers concise and jargon-free	Readability
End every response with a disclaimer	Compliance / assignment requirement

The retrieved context is injected directly into the system message as {context}, placing it at the highest-priority position in the prompt rather than as a separate user message.

3.2 Prompt Template Structure



3.3 Temperature Control

4. Safety Measures and Validation

4.1 Guardrail System (src/utls.py)

Six guardrail categories are evaluated before any LLM call takes place:

Category	Detection Method	Response
Emergency	Keyword substring match (deliberately broad — life-safety priority)	Direct emergency-services instruction
Safety violation	Keyword substring match (explicit harmful intent)	Rejection
Greeting	Regex, anchored patterns (^hi\$, ^hello\$, etc.)	Warm intro, RAG skipped
Diagnosis request	Regex with a named condition list	Refer to a professional
Medication change	Regex with filler-word tolerance	Refer to a physician
Off-topic	Keyword substring (unambiguous non-health domains only)	Polite redirect
<i>Design principle: the guardrails prefer false negatives over false positives. A borderline question that slips through to the LLM is still handled by the system prompt rules — a wrongly blocked legitimate healthcare question is a worse outcome.</i>		

The medication and diagnosis guardrails use regular expressions rather than simple substring matching, to avoid false positives on common healthcare phrases. For example, each of the following is correctly allowed through:

- "What vitamins should I take?" — allowed, since this is not a medication-change request
- "Do I have to avoid dairy?" — allowed, since this is not a diagnosis request
- "I hurt my ankle playing football" — allowed, since this is not off-topic

4.2 Response Validation

After the LLM responds, `is_low_confidence_answer()` inspects the first 15 words of the response for phrases suggesting the model admitted it could not find an answer (for example, an opening such as "I couldn't find reliable information..."). When such a phrase is detected, the response is replaced with the standardized `FALLBACK_RESPONSE`. Limiting the check to the first 15 words prevents valid responses from being discarded simply because they contain hedging language mid-sentence.

4.3 Error Handling

All exceptions raised in the Streamlit UI are caught and replaced with a generic, user-friendly message. Raw Python exception text is never shown to the user. Errors are logged internally through Python's logging module.

4.4 Mandatory Disclaimer

System prompt rule 9 requires the LLM to append the following disclaimer to every medical response:

 Disclaimer

This information is for educational purposes only and is not a substitute for professional medical advice.

A static disclaimer banner is also permanently displayed at the top of the UI, independent of LLM behavior.

5. Assumptions Made During Development

5. Ollama is a local dependency. The system requires Ollama to be running locally to generate embeddings — a deliberate choice to eliminate embedding API costs and keep document data on-device. The `run.py` pre-flight check verifies Ollama is running before the app launches.
6. Confidence scoring is retrieval-based, not answer-grounded. The High / Medium / Low indicator reflects how closely the top retrieved chunk matched the query by cosine distance; it does not measure whether the LLM's answer is factually correct. A chunk can be semantically close while only tangentially relevant.
7. Memory is session-scoped and in-memory only. Conversation history is not persisted to a database, and restarting the app clears all history — appropriate for a demo or assessment context.
8. The knowledge base is static. No real-time medical data sources are queried; all answers are grounded in documents ingested at setup time. Adding new documents requires re-running `python ingest.py`.
9. The chatbot targets general health education, not clinical decision support. It is designed for users seeking general health information rather than for clinical use, and all guardrails and disclaimers reflect this scope.
10. PDF content quality varies. Some PDFs — particularly older WHO documents — use non-standard encodings that `pypdf` cannot fully parse, which generates harmless “Advanced encoding” warnings during ingestion. The readable portions of those documents are still indexed.

6. Tech Stack Rationale

Component	Choice	Rationale
LLM	Groq API — llama-3.3-70b-versatile	Free tier, very fast inference (under 1s), Meta's latest open-weight model, strong instruction following
Embeddings	nomic-embed-text via Ollama	Local execution with no API cost; 768-dimensional embeddings competitive with paid alternatives; using the same model at ingest and query time guarantees vector consistency
Vector DB	ChromaDB	Lightweight, file-persisted, no separate server process, cosine similarity built in, sufficient for a ~3,000-chunk knowledge base
Framework	LangChain LCEL	The prompt llm parser chain is readable and maintainable, with native integration for ChatGroq and prompt templates
UI	Streamlit	Preferred by the assignment; enables rapid prototyping; session state handles memory with no separate backend needed
Launcher	Custom run.py	Pre-flight checks (Python version, Ollama, API key, vector store) catch setup errors before they reach the user